

CODE PART:

- Sol ① Our sandbox model learned on neat and spaced out Western houses. On contrary, Lucknow has ultra dense, packed neighborhood with shared walls and random roofing materials which our model has never seen before.
- Sol ② My model detected 4638 buildings. It missed most buildings in the old, clustered neighborhood where the roofs blend into one another.
- Sol ③ Open buildings need massive supercomputers and enormous global AI model that is much more powerful than our model. Use pre-made data when you need quick accurate baseline for standard regions. Build your own model when you are dealing with custom drone data, private imagery or need a real time tracking.

GUIDE PART

- Sol ① Satellite files for city takes up too much storage (gigabytes/terabytes) so, downloading it live by bounding box keeps the notebook light and fast. We can also change coordinates whenever we want in it.
- Sol ② a) Tiles become four times.
b) Clarity becomes more better, making small roofs and alleys super crisp.
- Sol ③ If we lower the confidence to 0.4, it gives a massive count but adds a lot of fake buildings. If we raise it to 0.9, it lowers the count but misses a lot of weird shaped houses.
- Sol ⑤ Instead of starting from scratch, the model already knows how to find basic things like lines, shapes and shadows. Fine tuning just takes those existing skills and tweaks them to recognize Indian-style roofs. This is
- Sol ⑥ MAP is a strict mathematical overlap test. If the model finds the building perfectly but its box is slightly tighter or looser than the human label, the math gives it a zero score even though the visual guess is great.
- Sol ⑦ A building count treats a 20 story apartment exactly same as a tiny one story house because it misses height and volume.
- Sol ⑧ Knowing how to build our own pipeline is also a lifesaver as in emergency situations because we can instantly fly a drone, grab fresh pictures, and find buildings in real time waiting for anyone else to update the data.

best_india.pt — 300 buildings in densest tile

